



Structured Query Language (SQL)

CSC1022 Introduction to Databases

CSC1005 Data Science & Databases
2025/2026 Semester 2

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Prof. Mark Roantree
Uttaran Bera
School of Computing
@ Dublin City University

Agenda



- 1 [Introduction](#)
- 2 [Simple Queries](#)
 - [Sorting Results](#)
- 3 [Aggregate Functions](#)
- 4 [Grouping Results](#)
- 5 [Subqueries](#)
- 6 [ANY and ALL](#)
- 7 [Multi-Table Queries](#)
- 8 [EXISTS](#)
- 9 [Combining Result Tables](#)
- 10 [Database Updates](#)

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

- How to extract datasets from a (potentially) large database
- Simple Queries
- What are aggregate functions?
- The power of sub-queries
- How to Join datasets



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- SQL is a declarative language where the user specifies what the result is to be and *not* how to compute it.
- The name SQL is presently expanded as Structured Query Language.
- SQL has a Data Manipulation Language (DML) component.
- SQL has a Data Definition Language (DDL) component.

DreamHome Case Study

Schema of all the tables needed for the following examples

```
Branch (branchNo, street, city, postcode)
Staff (staffNo, fName, lName, position, sex, DOB, salary, branchNo)
PropertyForRent (propertyNo, street, city, postcode,
type, rooms, rent, ownerNo, staffNo, branchNo)
Client (clientNo, fName, lName, telNo, prefType, maxRent)
PrivateOwner (ownerNo, fName, lName, address, telNo)
(clientNo, propertyNo, viewDate, comment)
```



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

- **SELECT:** to query data in the database;
- **INSERT:** to insert data into a table;
- **UPDATE:** to update data in a table;
- **DELETE:** to delete data from a table.



The **SELECT** statement is to retrieve data, powerful command capable of performing the equivalent of the relational algebra's *Selection*, *Projection*, and *Join* operations in a single statement.

```
SELECT      [DISTINCT | ALL] { * | [columnExpression [AS newName]] [, ...] }  
FROM      TableName [alias] [, ... ]  
[WHERE      condition]  
[GROUP BY  columnList] [HAVING condition]  
[ORDER BY  columnList]
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



The sequence of processing in a SELECT statement is:

FROM specifies the table or tables to be used
WHERE filters the rows subject to some condition
GROUP BY forms groups of rows with the same column value
HAVING filters the groups subject to some condition
SELECT specifies which columns are to appear in the output
ORDER BY specifies the order of the output

- The order of the clauses in the SELECT statement cannot be changed.
- The only two mandatory clauses are the first two: SELECT and FROM; the remainder are optional.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Retrieve all columns, all rows



Example 1

List full details of all staff

Since there are no restrictions specified in this query, the WHERE clause is unnecessary and all columns are required.

```
SELECT staffNo, fName, lName, position, sex, DOB, salary,  
       branchNo  
FROM Staff;
```

There is a far simpler way:

```
SELECT *  
FROM Staff;
```

Retrieve all columns, all rows: Results



Example 1

Table showing all details

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000.00	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000.00	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000.00	B005

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 2

Produce a list of salaries for all staff, showing only the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary
FROM Staff;
```

In this example, a new table is created from *Staff* containing only the designated columns *staffNo*, *fName*, *lName*, and *salary*, in the specified order.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

Retrieve specific columns, all rows: Results

Example 2 Results

Results showing specific columns

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG37	Ann	Beech	12000.00
SG14	David	Ford	18000.00
SA9	Mary	Howe	9000.00
SG5	Susan	Brand	24000.00
SL41	Julie	Lee	9000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 3

List the property numbers of all properties that have been viewed.

```
SELECT propertyNo FROM  
PropertyForRent ;
```

A different version with the results on the next slide.

```
SELECT DISTINCT propertyNo  
FROM PropertyForRent;
```

Retrieve specific columns, all rows: Results

Example 3 Results

Results showing specific columns and rows

There are duplicates because unlike the relational algebra *Projection* operation, SELECT does not eliminate duplicates when it *projects* over one or more columns.

propertyNo
PA14
PG4
PG4
PA14
PG36

propertyNo
PA14
PG4
PG36

To eliminate the duplicates, we use the **DISTINCT** keyword.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 4

Produce a list of monthly salaries for all staff, showing the staff number, the first and last names, and the salary details.

```
SELECT staffNo, fName, lName, salary/12  
FROM Staff;
```

This is an example of the use of a calculated field (sometimes called a computed or derived field).

Variation:

```
SELECT staffNo, fName, lName, salary/12 AS monthlySalary  
FROM Staff;
```

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Example 4 Results

- A SQL expression can use addition, subtraction, multiplication and division. Parentheses are used to build complex expressions.
- More than one table column can be used in a calculated column but columns referenced in an arithmetic expression *must* have a numeric type.

staffNo	fName	lName	col4
SL21	John	White	2500.00
SG37	Ann	Beech	1000.00
SG14	David	Ford	1500.00
SA9	Mary	Howe	750.00
SG5	Susan	Brand	2000.00
SL41	Julie	Lee	750.00

Normally, a calculated column takes its name from the corresponding column from which it has been retrieved: here SQL does not know how to label the column.



To restrict the rows to be retrieved, use the WHERE clause, followed by a search condition that specifies the rows to be retrieved.

- *Comparison.* Compare the value of one expression to the value of another expression.
- *Range.* Test whether the value of an expression falls within a specified range of values.
- *Set membership.* Test whether the value of an expression equals one of a set of values.
- *Pattern match.* Test whether a string matches a specified pattern.
- *Null.* Test whether a column has a null (unknown) value.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 5

List all staff with salary greater than 10000

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > 10000;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 5 Results

Results showing staff with salary greater than 10000

Here, the table is Staff and the predicate is *salary* > 10000.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG37	Ann	Beech	Assistant	12000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

The selection creates a new table containing only those Staff rows with a salary greater than 10,000.

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Comparisons: Usual Rules Apply!



=	equals		
< >	is not equal to (ISO standard)	!=	is not equal to (allowed in some dialects)
<	is less than	<=	is less than or equal to
>	is greater than	>=	is greater than or equal to

More complex predicates can be generated using the logical operators **AND**, **OR**, and **NOT**, with parentheses (if needed or desired) to show the order of evaluation. The rules for evaluating a conditional expression are:

- an expression is evaluated left to right;
- subexpressions in brackets are evaluated first;
- NOTs are evaluated before ANDs and ORs;
- ANDs are evaluated before ORs.

The use of parentheses is always recommended in order to remove any possible ambiguities.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

Example 6

List the addresses of all branch offices in London or Glasgow

```
SELECT *  
FROM Branch  
WHERE city = 'London' OR city = 'Glasgow';
```

Example 6 Results

Results showing offices in London or Glasgow

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B003	163 Main St	Glasgow	G11 9QX
B002	56 Clover Dr	London	NW10 6EU

The logical operator OR is used in the WHERE clause to find branches in London (city = 'London') *or* in Glasgow (city = 'Glasgow').

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Range search condition (BETWEEN/NOT BETWEEN)



Example 7

List all staff with a salary between 20,000 and 30,000

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary BETWEEN 20000 AND 30000;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 3 Results

Results showing staff with a specific salary range

The BETWEEN test includes the endpoints of the range, so any members of staff with a *salary* of 20,000 or 30,000 is included in the result.

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG5	Susan	Brand	Manager	24000.00

There is also a negated version of the range test (NOT BETWEEN) that checks for values outside the range.

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary >= 20000 AND salary <= 30000;
```

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Set membership search condition (IN/NOT IN)

Example 8

List all managers and supervisors

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position IN ('Manager', 'Supervisor');
```

We could have expressed the above query as:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE position = 'Manager' OR
      position = 'Supervisor';
```

However, the IN test provides a more efficient way of expressing the search condition, particularly if the set contains many values.



Example 8 Results

Results showing managers and supervisors only

staffNo	fName	lName	position
SL21	John	White	Manager
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Pattern match search condition (LIKE/NOT LIKE)

Example 9

Find all owners with the string 'Glasgow' in their address

For this query, we must search for the string 'Glasgow' appearing somewhere within the address column of the *PrivateOwner* table. SQL has two special pattern-matching symbols:

- % percent character represents any sequence of zero or more characters (wildcard).
- _ underscore character represents any single character.

```
SELECT ownerNo, fName, lName, address, telNo
FROM PrivateOwner
WHERE address LIKE '%Glasgow%';
```



Example 9 Results

Showing all owners with 'Glasgow' in their address

ownerNo	fName	lName	address	telNo
CO87	Carol	Farrel	6 Achray St, Glasgow G32 9DX	0141-357-7419
CO40	Tina	Murphy	63 Well St, Glasgow G42	0141-943-1728
CO93	Tony	Shaw	12 Park Pl, Glasgow G4 0QR	0141-225-7025

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result Tables](#)[Database Updates](#)



Example 10: Single Column Ordering

Produce a list of salaries for all staff, arranged in descending order of salary

```
SELECT staffNo, fName, lName, salary
FROM Staff
ORDER BY salary DESC;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 10 Results

List of staff with descending order of salary

Ordering is achieved by adding the ORDER BY clause to the end of the SELECT statement, specifying salary as the column to be sorted, and DESC to indicate that the order is to be descending.

staffNo	fName	lName	salary
SL21	John	White	30000.00
SG5	Susan	Brand	24000.00
SG14	David	Ford	18000.00
SG37	Ann	Beech	12000.00
SA9	Mary	Howe	9000.00
SL41	Julie	Lee	9000.00

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result Tables](#)[Database Updates](#)



Example 11

Produce an abbreviated list of properties arranged in order of property type

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 11 Results

Properties arranged in order by property type

propertyNo	type	rooms	rent
PL94	Flat	4	400
PG4	Flat	3	350
PG36	Flat	3	375
PG16	Flat	4	450
PA14	House	6	650
PG21	House	5	600

- There are four flats in this list.
- As we did not specify any minor sort key, the system arranges these rows in any order it chooses.
- To arrange the properties in order of rent, we specify a minor order.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

Example 11 Results

Properties arranged in order by property type and rent

propertyNo	type	rooms	rent
PG16	Flat	4	450
PL94	Flat	4	400
PG36	Flat	3	375
PG4	Flat	3	350
PA14	House	6	650
PG21	House	5	600

```
SELECT propertyNo, type, rooms, rent
FROM PropertyForRent
ORDER BY type, rent DESC;
```



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- COUNT returns the number of values in a specified column;
- SUM returns the sum of the values in a specified column;
- AVG returns the average of the values in a specified column;
- MIN returns the smallest value in a specified column;
- MAX returns the largest value in a specified column.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- These functions operate on a single column of a table and return a single value.
- COUNT, MIN, and MAX apply to both numeric and non-numeric fields, but SUM and AVG may be used on numeric fields only.
- Apart from COUNT(*), each function eliminates nulls first and operates only on the remaining non-null values.
- COUNT(*) is a special use of COUNT, which counts all the rows of a table, regardless of whether nulls or duplicate values occur.
- If we want to eliminate duplicates before the function is applied, we use the keyword DISTINCT before the column name in the function.



Example 13

How many properties cost more than 350 per month to rent?

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 13

How many properties cost more than 350 per month to rent?

```
SELECT COUNT(*) AS myCount  
FROM PropertyForRent  
WHERE rent > 350;
```

myCount
5

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Using COUNT(DISTINCT)



Example 14

How many different properties were viewed in May 2004?

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Using COUNT(DISTINCT)



Example 14

How many different properties were viewed in May 2004?

```
SELECT COUNT(DISTINCT propertyNo) AS myCount  
FROM  
WHERE viewDate BETWEEN '1-May-04' AND '31-May-04';
```

myCount
2

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 15

Find the total number of Managers and the sum of their salaries

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 15

Find the total number of Managers and the sum of their salaries

```
SELECT COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
WHERE position = 'Manager';
```

myCount	mySum
2	54000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 16

Find the minimum, maximum and average staff salary

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 16

Find the minimum, maximum and average staff salary

```
SELECT MIN(salary) AS myMin, MAX(salary) AS myMax, AVG(salary) AS  
       myAvg  
FROM Staff;
```

myMin	myMax	myAvg
9000.00	30000.00	17000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- A query with the GROUP BY clause is a *grouped* query, because it groups data from SELECT to produce **a single summary row** for each group.
- When GROUP BY is used, each item in the SELECT list must be **single-valued** per group.
- The SELECT clause contains only: column names; aggregate functions; constants; or any combination.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- A query with the GROUP BY clause is a *grouped* query, because it groups data from SELECT to produce a **single summary row** for each group.
- When GROUP BY is used, each item in the SELECT list must be **single-valued** per group.
- The SELECT clause contains only: column names; aggregate functions; constants; or any combination.
- All column names in the SELECT list **must appear** in the GROUP BY clause unless that column is used in an aggregate function.
- When the WHERE clause is used with GROUP BY, the WHERE clause is applied first, then groups are formed from the remaining rows that satisfy the search condition.



Example 17

Find the number of staff working in each branch and the sum of their salaries

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 17

Find the number of staff working in each branch and the sum of their salaries

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum  
FROM Staff  
GROUP BY branchNo  
ORDER BY branchNo;
```

- It is not necessary to include the column names *staffNo* and *salary* in the GROUP BY list because they appear only in the SELECT list within aggregate functions.
- On the other hand, *branchNo* is not associated with an aggregate function and so must appear in the GROUP BY list.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 17 Results

Result showing staff working in each branch and salaries

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00
B007	1	9000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

- SQL divides the staff into groups according to their respective branch numbers.
- Within each group, all staff have the same branch number.
- In this example, we get three groups:

branchNo	staffNo	salary		COUNT(staffNo)	SUM(salary)
B003	SG37	12000.00	}	3	54000.00
B003	SG14	18000.00			
B003	SG5	24000.00			
B005	SL21	30000.00	}	2	39000.00
B005	SL41	9000.00			
B007	SA9	9000.00	}	1	9000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- 1 For each group, SQL computes the *number* of staff members and calculates the *sum* of the values in the salary column to get the total of their salaries.
- 2 SQL generates a single summary row in the query result for each group.
- 3 Finally, the result is sorted in ascending order of branch number, *branchNo*.

Restricting groupings (HAVING clause)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- The HAVING clause is designed for use with the GROUP BY clause to restrict the groups that appear in the final result table.
- Although similar in syntax, HAVING and WHERE serve different purposes.
- The WHERE clause filters *individual rows*, whereas HAVING filters *groups*, going into the final result table.
- Column names used in the HAVING clause must also appear in the GROUP BY list or be contained within an aggregate function.



Example 18

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 18

For each branch office with more than one member of staff, find the number of staff working in each branch and the sum of their salaries.

```
SELECT branchNo, COUNT(staffNo) AS myCount, SUM(salary) AS mySum
FROM Staff
GROUP BY branchNo
HAVING COUNT(staffNo) > 1
ORDER BY branchNo;
```

branchNo	myCount	mySum
B003	3	54000.00
B005	2	39000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 19

List the staff who work in the branch at '163 Main St'

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = (SELECT branchNo
FROM Branch
WHERE street = '163 Main St');
```

The inner *SELECT* statement (*SELECT branchNo FROM Branch . . .*)

finds the branch number that corresponds to the branch with street name '163 Main St'.

There will be only one such branch number, so this is an example of a scalar subquery (which allows '=').



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Using a subquery with equality

Example 19

Staff who work in the branch at '163 Main St'

staffNo	fName	lName	position
SG37	Ann	Beech	Assistant
SG14	David	Ford	Supervisor
SG5	Susan	Brand	Manager

Having obtained this branch number, the outer SELECT statement then retrieves the details of all staff who work at this branch.

In other words, the inner SELECT returns a result table containing a single value 'B003', corresponding to the branch at '163 Main St', and the outer SELECT becomes:

```
SELECT staffNo, fName, lName, position
FROM Staff
WHERE branchNo = 'B003';
```

Using a subquery with an aggregate function

Example 20

List all staff whose salary is greater than the average salary, and show by how much their salary is greater than the average



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)

Using a subquery with an aggregate function



Example 20

List all staff whose salary is greater than the average salary, and show by how much their salary is greater than the average

```
SELECT staffNo, fName, lName, position,  
salary - (SELECT AVG(salary) FROM Staff) AS salDiff  
FROM Staff  
WHERE salary > (SELECT AVG(salary) FROM Staff);
```

- First, note that we cannot write **'WHERE salary > AVG(salary)'** because aggregate functions cannot be used in the **WHERE** clause.
- We must use a *subquery* to find the average salary, and then use the outer **SELECT** to find staff with a salary *greater* than this average.
- In other words, the subquery returns the average salary as 17,000.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Using a subquery with equality

Example 20 Results

All staff whose salary is greater than the average salary, and by how much their salary is greater than the average

staffNo	fName	lName	position	salDiff
SL21	John	White	Manager	13000.00
SG14	David	Ford	Supervisor	1000.00
SG5	Susan	Brand	Manager	7000.00

Note also the use of the scalar subquery in the SELECT list, to determine the difference from the average salary. The outer query is reduced then to:

```
SELECT staffNo, fName, lName, position, salary-17000 AS salDiff
FROM Staff
WHERE salary > 17000;
```



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Subquery Logic

The following rules apply to subqueries:

- 1 The ORDER BY clause may not be used in a subquery (only in the outermost SELECT).
- 2 The subquery SELECT list must consist of a single column name or expression, except for subqueries that use the keyword EXISTS
- 3 By default, column names in a subquery refer to the table name in the FROM clause **of the subquery**. It is possible to refer to a table in a FROM clause of **an outer query** by *qualifying* the column name.
- 4 When a subquery is one of the two operands involved in a comparison, the subquery must appear on the right-hand side of the comparison. As the subquery appears on the left-hand side of the comparison with *salary*, it would be incorrect as:

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE (SELECT AVG(salary) FROM Staff) < salary;
```

Nested Subqueries using IN



Example 21

List the properties that are handled by staff who work in the branch at '163 Main St'

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 21

List the properties that are handled by staff who work in the branch at '163 Main St'

```
SELECT propertyNo, street, city, postcode, type, rooms, rent
FROM PropertyForRent
WHERE staffNo IN
( SELECT staffNo
  FROM Staff
  WHERE branchNo = (SELECT branchNo
                    FROM Branch
                    WHERE street = '163 Main St') );
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



- Working from the innermost query: the first query selects the *branch number* at '163 Main St'.
- The second query selects those *staff* who work at this branch number.
- There may be more than one row found so we cannot use the equality condition (=) in the outermost query.
- Instead, we use the **IN keyword**.
- The outermost query then retrieves the details of the properties that are managed by each member of staff identified in the middle query.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 21 Results

The properties that are handled by staff who work in the branch at '163 Main St'

propertyNo	street	city	postcode	type	rooms	rent
PG16	5 Novar Dr	Glasgow	G12 9AX	Flat	4	450
PG36	2 Manor Rd	Glasgow	G32 4QX	Flat	3	375
PG21	18 Dale Rd	Glasgow	G12	House	5	600

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)



- The words ANY (SOME) and ALL may be used with subqueries that produce a single column of numbers.
- If the subquery is preceded by the keyword ALL, the condition will only be true if it is satisfied by *all* values produced by the subquery.
- If the subquery is preceded by the keyword ANY, the condition will be true if it is satisfied by *any* (one or more) values produced by the subquery.
- If the subquery is empty, the ALL condition returns true, the ANY condition returns false.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 22

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 22

Find all staff whose salary is larger than the salary of at least one member of staff at branch B003

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > SOME (SELECT salary
FROM Staff
WHERE branchNo = 'B003');
```

This query could also be expressed using a subquery that finds the minimum salary of the staff at branch B003, and then an outer query that finds all staff whose salary is greater than this number.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 22 Results

All staff whose salary is larger than the salary of at least one member of staff at branch B003

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00
SG14	David	Ford	Supervisor	18000.00
SG5	Susan	Brand	Manager	24000.00

The inner query produces the set 12000, 18000, 24000 and the outer query selects those staff whose salaries are greater than any of the values in this set (that is, greater than the minimum value, 12000).

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Example 23

Find all staff whose salary is larger than the salary of **every** member of staff at branch B003



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 23

Find all staff whose salary is larger than the salary of **every** member of staff at branch B003

```
SELECT staffNo, fName, lName, position, salary
FROM Staff
WHERE salary > ALL (SELECT salary
FROM Staff
WHERE branchNo = 'B003');
```

staffNo	fName	lName	position	salary
SL21	John	White	Manager	30000.00

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- To combine columns from several tables into a result table we need to use a join operation.
- The SQL join operation combines information from two tables by forming pairs of related rows from the two tables.
- The row pairs that make up the joined table are those where the matching columns in each of the two tables have the same value.

Example 24

List the names of all clients who have viewed a property along with any comment supplied



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 24

List the names of all clients who have viewed a property along with any comment supplied

```
SELECT c.clientNo, fName, lName, propertyNo, comment
FROM Client c, v
WHERE c.clientNo = v.clientNo;
```

Note that it is necessary to qualify the client number *clientNo*, in the SELECT list: *clientNo* could come from either table, and we have to indicate which one.

The qualification is achieved by prefixing the column name with the appropriate table name (or its alias).

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Join Results

Example 24 Results

Names of all clients who have viewed a property along with any comment supplied

- To obtain the required rows, we include those rows from both tables that have identical values in the *clientNo* columns, using the search condition *c.clientNo* = *v.clientNo*.
- We call these two columns the *matching* columns for the two tables.

clientNo	fName	lName	propertyNo	comment
CR56	Aline	Stewart	PG36	
CR56	Aline	Stewart	PA14	too small
CR56	Aline	Stewart	PG4	
CR62	Mary	Tregear	PA14	no dining room
CR76	John	Kay	PG4	too remote



Example 25

For each branch office, list the numbers and names of staff who manage properties and properties they manage.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 25

For each branch office, list the numbers and names of staff who manage properties and properties they manage.

```
SELECT s.branchNo, s.staffNo, fName, lName, propertyNo
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
ORDER BY s.branchNo, s.staffNo, propertyNo;
```

We have ordered the output using the branch number as the major sort key and the staff number and property number as the minor keys.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 25

For each branch office, the numbers and names of staff who manage properties and properties they manage.

branchNo	staffNo	fName	lName	propertyNo
B003	SG14	David	Ford	PG16
B003	SG37	Ann	Beech	PG21
B003	SG37	Ann	Beech	PG36
B005	SL41	Julie	Lee	PL94
B007	SA9	Mary	Howe	PA14

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 26

For each branch office, list the numbers and names of staff who manage properties, including the city in which the branch is located and the properties they manage.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 26

For each branch office, list the numbers and names of staff who manage properties, including the city in which the branch is located and the properties they manage.

```
SELECT b.branchNo, b.city, s.staffNo, fName, lName, propertyNo
FROM Branch b, Staff s, PropertyForRent p
WHERE b.branchNo = s.branchNo AND s.staffNo = p.staffNo
ORDER BY b.branchNo, s.staffNo, propertyNo;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 26 Results

The result table requires columns from three tables: *Branch*, *Staff*, and *PropertyForRent*, so a join must be used.

branchNo	city	staffNo	fName	lName	propertyNo
B003	Glasgow	SG14	David	Ford	PG16
B003	Glasgow	SG37	Ann	Beech	PG21
B003	Glasgow	SG37	Ann	Beech	PG36
B005	London	SL41	Julie	Lee	PL94
B007	Aberdeen	SA9	Mary	Howe	PA14

- The *Branch* and *Staff* details are joined using the condition *b.branchNo* = *s.branchNo*, to link each branch to the staff who work there.
- The *Staff* and *PropertyForRent* details are joined using the condition *s.staffNo* = *p.staffNo*, to link staff to the properties they manage.

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)



Example 27

Find the number of properties handled by each staff member

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 27

Find the number of properties handled by each staff member

- To list the required numbers, we first need to find out which staff actually manage properties.
- This can be found by joining the Staff and PropertyForRent tables on the staffNo column, using the FROM/WHERE clauses.
- Next, we need to form groups consisting of the branch number and staff number, using the GROUP BY clause.
- Finally, we sort the output using the ORDER BY clause.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 27

Find the number of properties handled by each staff member

- To list the required numbers, we first need to find out which staff actually manage properties.
- This can be found by joining the Staff and PropertyForRent tables on the staffNo column, using the FROM/WHERE clauses.
- Next, we need to form groups consisting of the branch number and staff number, using the GROUP BY clause.
- Finally, we sort the output using the ORDER BY clause.

```
SELECT s.branchNo, s.staffNo, COUNT(*) AS myCount
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.branchNo, s.staffNo
ORDER BY s.branchNo, s.staffNo;
```



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 27 Results

Find the number of properties handled by each staff member

The result table requires columns from three tables: Branch, Staff, and PropertyForRent, so a join must be used.

branchNo	staffNo	myCount
B003	SG14	1
B003	SG37	2
B005	SL41	1
B007	SA9	1

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

- 1 Form the Cartesian product of the tables named in the FROM clause.
- 2 If there is a WHERE clause, apply the search condition to each row of the product table, retaining those rows that satisfy the condition. In terms of the relational algebra, this operation yields a restriction of the Cartesian product.
- 3 For each remaining row, determine the value of each item in the SELECT list to produce a single row in the result table.
- 4 If SELECT DISTINCT has been specified, eliminate any duplicate rows from the result table. In the relational algebra, Steps 3 and 4 are equivalent to a projection of the restriction over the columns mentioned in the SELECT list.
- 5 If there is an ORDER BY clause, sort the result table as required.

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Outer joins

- The **Join** operation combines data from two tables by forming pairs of related rows where the matching columns in each table have the same value.
- If one row of a table is unmatched, the row is omitted from the result table.
- The **Outer Join** retains rows that do not satisfy the join condition.
- To understand the Outer join operators, consider the following two simplified *Branch* and *PropertyForRent* tables, which we refer to as *Branch1* and *PropertyForRent1*.

Branch1

branchNo	bCity
B003	Glasgow
B004	Bristol
B002	London

PropertyForRent1

propertyNo	pCity
PA14	Aberdeen
PL94	London
PG4	Glasgow

Example 28

Inner Join Example

```
SELECT b.*, p.*  
FROM Branch1 b, PropertyForRent1 p  
WHERE b.bCity = p.pCity;
```

The result table has two rows where the cities are the same.

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



Example 29

List **all branch offices** and any properties that are in the same city

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 29

List **all branch offices** and any properties that are in the same city

```
SELECT b.*, p.*  
FROM Branch1 b LEFT JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 29 Results

All branch offices and any properties that are in the same city

The Left Outer join includes not only those rows that have the same city, but also those rows of the first (left) table that are unmatched with rows from the second (right) table.

branchNo	bCity	propertyNo	pCity
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

The columns from the second table are filled with NULLs.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 30

List **all properties** and any branch offices that are in the same city

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 30

List **all properties** and any branch offices that are in the same city

```
SELECT b.*, p.*  
FROM Branch1 b RIGHT JOIN PropertyForRent1 p  
ON b.bCity = p.pCity;
```

Example 30 Results

All properties and any branch offices that are in the same city

The Right Outer join includes not only those rows that have the same city, but also those rows of the second (right) table that are unmatched with rows from the first (left) table.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B002	London	PL94	London

The columns from the first table are filled with NULLs.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



Example 31

List the branch offices and properties that are in the same city along with any unmatched branches or properties

```
SELECT b.*, p.*  
FROM Branch1 b FULL JOIN PropertyForRent1 p ON b.bCity = p.pCity;
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Full Outer join

Example 31 Results

Branch offices and properties that are in the same city

The Full Outer join includes not only those rows that have the same city, but also those rows that are unmatched in both tables.

branchNo	bCity	propertyNo	pCity
NULL	NULL	PA14	Aberdeen
B003	Glasgow	PG4	Glasgow
B004	Bristol	NULL	NULL
B002	London	PL94	London

The unmatched columns are filled with NULLs.



- The keywords EXISTS and NOT EXISTS are designed for use only with subqueries.
- They produce a simple true/false result.
- EXISTS is true if and only if there exists at least one row in the result table returned by the subquery; it is false if the subquery returns an empty result table.
- NOT EXISTS is the opposite of EXISTS.
- Since EXISTS and NOT EXISTS check only for the existence or non-existence of rows in the subquery result table, the subquery can contain any number of columns.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 32

Find all staff who work in a London branch office

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 32

Find all staff who work in a London branch office

```
SELECT staffNo, fName, lName, position
FROM Staff s
WHERE EXISTS (SELECT *
FROM Branch b
WHERE s.branchNo = b.branchNo AND city = 'London');
```

- This query could be rephrased as: *Find all staff such that there exists a Branch row containing his/her branch number, branchNo, and the branch city equal to London.*
- The test for inclusion is the *existence* of such a row.
- If it exists, the subquery evaluates to *true*.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 32 Results

All staff who work in a London branch office

staffNo	fName	lName	position
SL21	John	White	Manager
SL41	Julie	Lee	Assistant

Note that the first part of the search condition, *s.branchNo* = *b.branchNo*, is necessary to ensure that we consider the correct branch row for each member of staff.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



If we omit this part of the query, we would get all staff rows listed because the subquery (*SELECT * FROM Branch WHERE city = 'London'*) would always be true and the query would be reduced to:

```
SELECT staffNo, fName, lName, position FROM Staff WHERE true;
```

which is equivalent to:

```
SELECT staffNo, fName, lName, position FROM Staff;
```

We could also have written this query using the join construct:

```
SELECT staffNo, fName, lName, position  
FROM Staff s, Branch b  
WHERE s.branchNo = b.branchNo AND city = 'London';
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



In SQL, we can use the normal set operations of **Union**, **Intersection**, and **Difference** to combine the results of two or more queries into a single result table.

- The **Union** of two tables *A* and *B*, is a table containing all rows that are in either the first table *A* **or** the second table *B* or both.
- The **Intersection** of two tables *A* and *B*, is a table containing all rows that are **common to both** tables *A* and *B*.
- The **Difference** of two tables *A* and *B*, is a table containing all rows that are in table *A* but are **not** in table *B*.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

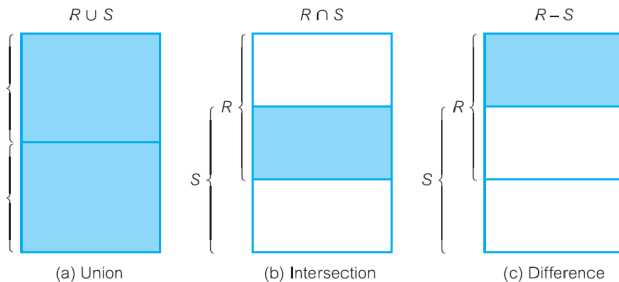
[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

The most important restriction is that the two tables must be union-compatible: they have the *same structure*.



This implies that the two tables must contain the same number of columns, and that their corresponding columns have the same data types and lengths.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result Tables](#)

[Database Updates](#)



Example 33

Construct a list of all cities where there is either a branch office or a property

```
(SELECT city
FROM Branch
WHERE city IS NOT NULL)
UNION
(SELECT city
FROM PropertyForRent
WHERE city IS NOT NULL);
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 33 Results

All cities where there is either a branch office or a property

This query is executed by:
producing a result table from the first query and a result table
from the second query,
and then merging all rows from both tables into a single table
with duplicate rows removed.

city
London
Glasgow
Aberdeen
Bristol

[Introduction](#)[Simple Queries](#)[Sorting Results](#)[Aggregate Functions](#)[Grouping Results](#)[Subqueries](#)[ANY and ALL](#)[Multi-Table Queries](#)[EXISTS](#)[Combining Result
Tables](#)[Database Updates](#)

Use of INTERSECT

Example 34

Construct a list of all cities where there is both a branch office or a property

This query is executed by producing a result table from the first and second queries, and then creating a single table consisting of those rows that are *common to both* tables.

```
(SELECT city FROM Branch)
INTERSECT
(SELECT city FROM PropertyForRent);
```

city
Aberdeen
Glasgow
London

Same query without INTERSECT

Example 34 (rewritten)

Construct a list of all cities where there is both a branch office or a property

```
SELECT DISTINCT b.city
FROM Branch b, PropertyForRent p
WHERE b.city = p.city;
```

Or as:

```
SELECT DISTINCT city
FROM Branch b
WHERE EXISTS (SELECT *
              FROM PropertyForRent p
              WHERE b.city = p.city);
```



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 35

Construct a list of all cities where there is a branch office but no properties

This query is executed by producing a result table from the first and second queries, and then creating a single table consisting of those rows that appear in the first table *but not* in the second.

```
(SELECT city FROM Branch)
EXCEPT
(SELECT city FROM PropertyForRent);
```

city
Bristol



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



SQL is a complete data manipulation language that can be used for modifying the data in the database as well as querying the database.

- INSERT adds new rows of data to a table
- UPDATE modifies existing data in a table
- DELETE removes rows of data from a table

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 36

Insert a new row into the Staff table supplying data for all columns

```
INSERT INTO Staff
VALUES ('SG16', 'Alan', 'Brown', 'Assistant', 'M',
DATE '1957-05-25', 8300, 'B003');
```

- As we are inserting data into each column in the order the table was created, there is no need to specify a column list.
- Note that character literals such as 'Alan' must be enclosed in single quotes.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 37

Insert a new row into the Staff table supplying data for all mandatory columns:

staffNo, fName, lName, position, salary and branchNo.

```
INSERT INTO Staff (staffNo, fName, lName, position, salary,  
                  branchNo)  
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', 8100, 'B003');
```

- As we are inserting data into certain columns only, we must specify the names of the target columns.
- The order for the column names is not significant, but it is usual to specify them in the order they appear in the table.

```
INSERT INTO Staff  
VALUES ('SG44', 'Anne', 'Jones', 'Assistant', NULL, NULL, 8100, '  
B003');
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 38

Populate StaffPropCount from Staff & PropertyForRent

Assume there is a table StaffPropCount containing names of staff and the number of properties they manage:

StaffPropCount(staffNo, fName, lName, propCount)

```
INSERT INTO StaffPropCount
(SELECT s.staffNo, fName, lName, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.staffNo, fName, lName)
UNION
(SELECT staffNo, fName, lName, 0
FROM Staff s
WHERE NOT EXISTS (SELECT *
                   FROM PropertyForRent p
                   WHERE p.staffNo = s.staffNo));
```

- If we omit the second part of the UNION, we get a list of staff who manage *at least* one property: it excludes staff who manage no properties.



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

Example 38 Logic



To include the staff who do not manage a property, use the UNION statement with a second SELECT to add these staff, using a 0 value for the count attribute.

staffNo	fName	lName	propCount
SG14	David	Ford	1
SL21	John	White	0
SG37	Ann	Beech	2
SA9	Mary	Howe	1
SG5	Susan	Brand	0
SL41	Julie	Lee	1

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

UPDATE all rows



Example 39

Give all staff a 3% pay increase

```
UPDATE Staff
SET salary = salary*1.03;
```

As the update applies to all rows in the Staff table, the WHERE clause is omitted.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 40

Give all managers a 5% pay increase

```
UPDATE Staff
SET salary = salary*1.05
WHERE position = 'Manager';
```

The WHERE clause finds the rows that contain data for Managers and the update salary = salary * 1.05 is applied only to these particular rows.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 41

Promote David Ford (staffNo = 'SG14') to Manager and change his salary to 18,000

```
UPDATE Staff
SET position = 'Manager', salary = 18000
WHERE staffNo = 'SG14';
```

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 42

Delete all rows related to property PG4

```
DELETE FROM  
WHERE propertyNo = 'PG4';
```

The WHERE clause finds the rows for property PG4 and the delete operation is applied only to these particular rows.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



Example 43

Delete all rows from the table

```
DELETE FROM ;
```

- No WHERE clause has been specified, so the delete operation applies to all rows in the table.
- This leaves an empty table (definition) so one can still insert data into the table at a later stage.

[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)



[Introduction](#)

[Simple Queries](#)

[Sorting Results](#)

[Aggregate Functions](#)

[Grouping Results](#)

[Subqueries](#)

[ANY and ALL](#)

[Multi-Table Queries](#)

[EXISTS](#)

[Combining Result
Tables](#)

[Database Updates](#)

